Pipeline Outputs Reference

This document describes **every file produced by the pipeline**, end to end: from the Stage 1 repo-analysis run (`run.py`) through the Stage 2 reporting scripts (`code/reports/`). For each file: what writes it, where it lands, and what every column/figure means.

Overview: two stages

1. **Stage 1 - Analysis (`run.py` + `repo_analyzer.py`)**: clones/queries each repo in a project list, detects AI coding-tool usage, and writes one working directory per repo plus experiment-level CSVs.
2. **Stage 2 - Reporting (`code/reports/*.py`)**: reads the Stage 1 output (plus an external project-metrics CSV like `data/projects-29-08.csv`), and produces LaTeX tables/macros and chart PDFs under `<experiment_dir>/report/`.

```
<output-dir>/<experiment_name>/            <- Stage 1 output root
+-- working/<owner>-<repo>/                <- staging dir while a repo is analyzed
+-- analysis_error/<owner>-<repo>/
+-- tool_commit_adopters/<owner>-<repo>/
+-- tool_only_adopters/<owner>-<repo>/
+-- ignored_tools_commit_adopters/<owner>-<repo>/
+-- commit_only_adopters/<owner>-<repo>/
+-- ignored_tools_only_adopters/<owner>-<repo>/
+-- non_adopters/<owner>-<repo>/
+-- analysis_report_<N>.csv
+-- retry_list_<N>.csv
+-- pull_requests_stats_<N>.csv
+-- overall_ratios_<N>.csv
+-- since_adoption_ratios_<N>.csv
+-- recent_adoption_ratios_<N>.csv
+-- report/                               <- Stage 2 output root (created by reports/common.py)
    +-- rq1/
    +-- rq2/
    +-- rq3/
    +-- rq4/
    +-- rq5/
    +-- extra/
```

Stage 1 - Analysis pipeline

Entry point: `run.py`

Input: a project-list CSV (`github_url` column, or `project_name`/`name` which gets turned into a URL), an experiment name, and an analysis start date.

Flow: read project list -> analyze each repo in parallel (thread/process pool) via `RepoAnalyzer.do_analysis()` -> move each repo's working directory into a classification subfolder -> write experiment-level CSVs.

Which subfolder a repo lands in (`get_classification_dir`, based on the repo's `AnalysisResult` flags):

Folder	Condition
`analysis_error/`	`error` is non-empty (analysis crashed)
`tool_commit_adopters/`	tool files found **and** AI-authored commits found
`tool_only_adopters/`	tool files found, no AI-commit evidence

Folder	Condition
`ignored_tools_commit_adopters/`	tool files only present via `.gitignore` **and** AI-authored commits found
`commit_only_adopters/`	AI-authored commits found, no tool-file evidence
`ignored_tools_only_adopters/`	tool files only via `.gitignore`, no commit evidence
`non_adopters/`	none of the above

Experiment-level CSV files (written in `<output-dir>/<experiment_name>/`)

`analysis_report_<N>.csv` / `tmp_analysis_<N>.csv` (successes) and `retry_list_<N>.csv` (errored rows)

One row per repo. Columns (`AnalysisResult`):

Column	Type	Meaning
`github_url`	str	repo's GitHub URL
`path`	str	filesystem path to the repo's working directory (after classification move)
`log_path`	str	path to the repo's `analysis.log`
`tool_use`	bool	an AI-tool config/marker file (e.g. `.cursor/`, `CLAUDE.md`) was found in the repo
`commit_use`	bool	the repo's overall AI-commit ratio is > 0
`ignored_tools`	bool	tool marker files exist only inside `.gitignore` (tool used but files never committed)
`adoption_date`	datetime/None	earliest timestamp across all detected tool files/AI commits - the inferred "first AI adoption" moment
`last_file_change`	datetime/None	most recent modification timestamp among detected tool config files
`ratio`	float/None	overall AI-commit ratio (%) across the whole analysis period
`adoption_ratio`	float/None	AI-commit ratio (%) computed only from the adoption date onward
`since_adoption_commits`	int/None	number of commits counted in the "since adoption" window
`recent_ratio`	float/None	AI-commit ratio (%) in the most recent window (last 60 days, or since adoption if later)
`recent_commits`	int/None	commit count in that recent window
`sampled`	bool	this repo's URL was present in the `sample_list`
`duration`	timedelta	wall-clock time taken to analyze this repo
`error`	str	traceback text if analysis failed; empty string on success

`pull_requests_stats_<N>.csv`

One row per repo. Columns (`PullRequestStats`):

Column	Type	Meaning
`total_prs`	int	total PRs analyzed (`-1` if analysis never ran)
`counts`	dict (JSON)	number of PRs per merge-type classification
`percent`	dict (JSON)	percentage of PRs per merge-type classification
`large_pr_threshold`	int	commit-count threshold above which a PR counts as "large"
`total_large_prs`	int	number of PRs meeting/exceeding that threshold
`large_counts`	dict (JSON)	merge-type counts restricted to large PRs
`github_url`	str	repo this row belongs to

Merge-type values: `unmerged`, `merge`, `squash`, `rebase`, `partial-rebase`, `merge (fast-forward)`, `unknown`.

`overall_ratios_<N>.csv`, `since_adoption_ratios_<N>.csv`, `recent_adoption_ratios_<N>.csv`

One file per time-window "bucket"; one row per repo (`name` = the repo's `github_url` in these top-level files). Columns (`RatioMetrics`):

Column	Type	Meaning
`name`	str	repo's `github_url` (top-level files)
`type`	str	sub-bucket: `overall`/`default`, a file-type (`source_code`/`documentation`/`configuration`/`data`/`build`/`other`), or a language name
`period_start` / `period_end`	datetime/None	start/end of the time window this row covers
`commits`	int	total human+AI commits in this bucket (denominator for `commit_ratio`)
`commit_ratio`	float/None	% of commits authored by AI tools
`added_lines` / `deleted_lines`	int	total added/deleted lines (human+AI)
`added_lines_ratio` / `deleted_lines_ratio`	float/None	% of added/deleted lines attributable to AI commits
`add_minus_del`	int	total net (added - deleted) lines
`add_minus_del_ratio`	float/None	% share of net line delta from AI commits
`contributors`	int	distinct human+AI authors
`contributor_ratio`	float/None	% of contributors with at least one AI-tagged commit
`files`	int	distinct files touched
`files_ratio`	float/None	% of touched files touched by at least one AI commit

Per-repo working directory (inside each `<classification>/<owner>-<repo>/`)

File	Contents
`analysis.log`	per-repo log of the analysis run
`files.txt`	newline-delimited list of every file path in the repo
`commit_list.csv`	every commit (`LogCommit`): `sha, author, co_authors, message, date, parents, in_main_branch`

File	Contents
commit_stats_main.csv	per-commit classification cache (StatSummary = LogCommit + auth_type [Human/AI/Bot/AI-FOLLOWUP] + tool [which AI tool was detected])
commit_stats_files.csv	per-file-change cache (CachedFileStat): path, type, added_lines, deleted_lines, old_path, commit_sha (links back to commit_stats_main.csv via commit_sha)
adoption_dates.csv	one row per detected tool: tool, adoption (earliest timestamp that tool was observed)
ratios.csv	all RatioMetrics rows (every bucket x sub-type/language x period) for this repo - same columns as the experiment-level ratio files, but name is the bucket label (overall/since_adoption/recent_adoption/monthly)
clones/partial/	blob-less, no-checkout git clone (used for file-history metrics & commit log)
clones/shallow/	shallow-since git clone (used for full commit-stats numstat)
pull_requests/pr_list.csv	one row per PR (BasePullRequest): url, number, owner, repo, branch_name, title, bodyText, updated_at, merged_at, merge_commit_sha, additions, deletions, changed_files, labels, pr_author
pull_requests/pr_commits.csv	one row per PR commit (PRCommit): sha, author, co_authors, message, date, pr_number
tools/tool_files.csv	tool marker files found: tool, file_path
tools/tool_metrics.csv	per-matched-file metrics: tool, file_path, added_timestamp, last_modification, lines, modifications
tools/tool_ignored_files.csv	same shape, for tool files that only appear in .gitignore (never committed)
tools/_gitignore, tools/_gitignore_tool_matches_<N>	raw .gitignore content and matched-pattern snapshots
tools/<tool>_<encoded_path>	locally-saved copies of matched tool-config file blobs

Detection logic reference

- heuristics.py - regex pattern sets loaded from config/patterns/*.csv:
- authors.csv -> identifies AI-tool authorship by commit author name/email (Claude, ChatGPT, Copilot, Cursor, aider, Codex, Gemini, ...)
- files.csv -> identifies AI-tool config/marker files (.cursor/, CLAUDE.md, .aider.conf.yaml, AGENTS.md, ...)
- bots.csv -> non-AI automation bots (Dependabot, Renovate, GitHub Actions, ...), distinguished from AI tools
- labels.csv -> PR label text patterns indicating AI involvement
- branches.csv -> PR branch-name patterns indicating AI-generated branches
- utils/commit_type_classifier.py - classifies conventional-commit message prefixes into build, chore, ci, docs, feat, fix, perf, refactor, revert, style, test (or none). This is a commit-*type* taxonomy, unrelated to AI-authorship detection; used only by Stage 2 reports.

Utility scripts (not part of the per-run output, but touch the same files)

- get_pull_requests.py - fetches PRs via GitHub GraphQL; feeds pr_list.csv/pr_commits.csv. Its own __main__ demo (not used by the pipeline) can write an ad hoc github_prs.csv.
- clone_missing_repos.py - repair/backfill utility: re-clones/re-analyzes repos whose clones/shallow is missing or broken; can synthesize a fallback commit_stats_main.csv for repos that never went through full commit classification.

- `utils/cloning.py` - `clone_repo`, `move_directory_robust` (retries the classification-folder move on Windows file locks), `update_repo`.

Stage 2 - Reporting pipeline (`code/reports/`)

All report scripts share `reports/common.py`. Every script is invoked with a positional `input` (the Stage 1 experiment directory) and writes under `<input>/report/...`. Common CLI flags: `--select` / `--exclude` (comma-separated substrings to filter `github_url`).

Most scripts combine the Stage 1 `analysis_report_*.csv` with an external **project metrics CSV** (default `data/projects-29-08.csv`, columns: `project_name, github_url, age_years, contributors, commits, total_issues, total_pull_requests, code_lines, topics, classification, numerical_level`) via `merge_dataframes_by_github_url`. **This metrics file must actually contain those metric columns** - a plain repo-list CSV (just a `name`/`github_url` column, no metrics) will fail downstream with a `KeyError` on columns like `code_lines`.

`consolidate_metrics.py` - prerequisite for all RQ3 scripts

Joins every repo's `ratios.csv` with the metrics file, the Stage 1 report, and aggregated `tool_metrics.csv` data.

- **Output:** `rq3/consolidated_metrics_main.csv` (overall/code/documentation/configuration commit ratios & counts, files/contributor ratios, line ratios, velocity, plus tool-usage totals) and `rq3/consolidated_metrics_pl.csv` (same, but with per-programming-language ratio/count columns for ~49 languages).

`basic_stats.py` - RQ1: headline adoption rate

- **Input:** Stage 1 `analysis_report_*.csv`.
- **Output:** `rq1/statistics.tex` - LaTeX macros + a summary table of tool-use %, commit-use %, "ignored tools only" %, combined adoption %, and an extrapolated high-estimate overall adoption rate.

`table_ratios.py` - RQ2: adoption rate by project-size decile

- **Input:** Stage 1 report merged with the metrics file.
- **Output (default):** `rq2/ratio_tables.tex` (decile tables + sparklines for `code_lines`, `contributors`, `commits`, `total_issues`, `total_pull_requests`, `age_years` vs. `tool_use`) and `rq2/file_effect_sizes.tex` (chi-squared results).
- **Output (`--no-one-year` / `--no-two-year`):** `rq2/adoption_sparklines_oneplus.tex` / `..._twoplus.tex` (sparkline-only, excluding young repos).

`commit_ratio_spark.py` - RQ2: categorical adoption-ratio table

- **Input:** Stage 1 report merged with the metrics file, filtered to `since_adoption_commits > 10`.
- **Processing:** buckets `adoption_ratio` into None/Experimental/Limited/Consistent/Pervasive across 5 adoption-definition masks (file-based, commit-based, etc.), with decile sparklines by LOC/contributors/age.
- **Output:** `rq2/adoption_ratio_table.tex`, `rq2/commit_effect_sizes.tex`.

`tool_metrics_report.py` - RQ2: file-intensity vs. commit-ratio correlation (charts)

- **Input:** Stage 1 report + aggregated `tool_metrics.csv` (`total_files`, `total_lines`, `total_changes`).
- **Output:** `rq2/file_metrics_aggregated.csv` (merged data), `rq2/file_vs_commit_ratio_{001,002}.pdf` (binned histograms + a pairplot of Files/LOC/Changes/Commit-Ratio).

`tool_metrics_table_spark.py` - RQ2: same data, as a LaTeX table

- **Output:** `rq2/file_adoption_table.tex` (binned distribution table with sparklines for `total_files`, `total_lines`, `total_changes`, `adoption_ratio`).

`commit_ratio_report.py` - RQ3: size/activity vs. intensity distributions (charts)

- **Input:** `rq3/consolidated_metrics_main.csv` and `rq3/consolidated_metrics_pl.csv`.
- **Output:** `extra/commit_ratio_report_main.pdf` (violin plots, binned histograms, correlation heatmap) and `extra/commit_ratio_report_pl.pdf` (per-language binned histograms).

`ratio_pl_report.py` - RQ3: per-language adoption distribution (charts)

- **Input:** `rq3/consolidated_metrics_pl.csv`, filtered to `commit_use == True` and `adoption_ratio > 0`.
- **Output:** `rq3/ratio_category_report_{001..}.pdf` - box+jitter plots of commit-ratio per language (vs. the "code" reference) and per repo topic.
- Note: **Naming collision:** `ratio_category_report.py` (below) writes to the same `rq3/ratio_category_report.pdf` path - running both against the same experiment will overwrite one with the other.

`ratio_category_report.py` - RQ3: category adoption distribution vs. commit-count threshold (charts)

- **Input:** `rq3/consolidated_metrics_main.csv`, filtered to `commit_use == True` and `overall_r > 0`.
- **Output:** `rq3/ratio_category_report.pdf` - one page per commit-count threshold (0/10/20/50), bar chart of category-distribution delta from the `overall_r` reference.

`repo_properties_report.py` - RQ3: topics/languages vs. adoption rate (charts)

- **Input:** Stage 1 report merged with a richer metrics file (default `data/all-projects-labels-topics-languages.csv`, needs `topics`/`labels`/`languages` columns).
- **Output:** `rq3/all_repo_properties_report.pdf` - scatter plots of adoption rate (%) vs. project frequency, for topics and for significant programming languages.

`top_organizations.py` - RQ3: adoption rate by GitHub organization

- **Input:** Stage 1 report + metrics file + `data/org_repo_counts.csv` (org->group mapping).
- **Output:** `../temp/org_adoption.csv` (per-group adoption %/repo count - note: **not** under `report/`) and `rq3/organization_adoption.tex` (top-20-orgs LaTeX table + macros).

`commit_sizes.py` / `commit_sizes_new.py` - RQ5: commit size & type by author (Human/Bot/AI)

- **Input:** every repo's `commit_stats_files.csv` + `commit_stats_main.csv` (cached to a local CSV to skip recomputation).
- **Output:** `rq5/commit_sizes_report_{001..00N}.pdf` (boxplots, binned distributions, commit-type breakdowns by author category) and `rq5/commit_sizes_boxplot_stats.tex`. `commit_sizes_new.py` is the newer version: drops the non-adopter comparison, adds a temporal (adoption-period) breakdown, and writes to a caller-specified `--output-pdf` path instead of a hardcoded `rq5/` location.
- Console-only: Shapiro-Wilk/normaltest normality tests, Mann-Whitney U, Cliff's delta effect size (AI vs. Human commit sizes).

`timelines.py` - RQ4: adoption over time

- **Input:** Stage 1 report + every repo's `adoption_dates.csv` + `config/patterns/top-orgs.csv` + `data/org_repo_counts.csv`.

- **Output:** `rq4/timeline_report_{001..004}.pdf` (cumulative adoption curve; ridgeline plots by tool and by organization; UpSet plot of tool co-adoption) and `rq4/figure_counts.tex` (extracted statistics as LaTeX macros). Also caches a merged dataframe to `timeline_info.csv` in the current working directory (not under `report/`).

Practical notes

- **The metrics file matters.** Scripts like `table_ratios.py`, `commit_ratio_spark.py`, `top_organizations.py`, and `consolidate_metrics.py` all expect `--metric-file` to point at a CSV with real metric columns (`code_lines`, `contributors`, `commits`, `total_issues`, `total_pull_requests`, `age_years`, `topics`, ...) - e.g. `data/projects-29-08.csv`. A bare project-list file (just repo names/URLs, like `data/trial1.csv`) will merge successfully but produce a dataframe missing those columns, causing a `KeyError` downstream.
- **Run order for RQ3:** `consolidate_metrics.py` must run before `commit_ratio_report.py`, `ratio_pl_report.py`, or `ratio_category_report.py`, since those three read `rq3/consolidated_metrics_{main,pl}.csv` rather than the raw Stage 1 report.
- **Known output collision:** `ratio_pl_report.py` and `ratio_category_report.py` both write `rq3/ratio_category_report.pdf`.